Content-Aware Image Compression via LLM-Guided Semantic Segmentation

Shahzan Abbas Raza
School of Computer Engineering
KIIT Deemed University
Bhubaneswar, India
shahzanabbas29@gmail.com

Mobasshir Ajaz
School of Computer Engineering
KIIT Deemed University
Bhubaneswar, India
mobasshirajaz@gmail.com

Daithoulung Rongmai
School of Computer Engineering
KIIT Deemed University
Bhubaneswar, India
daithoulungrongmai@gmail.com

Pratyush Shrivastava
School of Computer Engineering
KIIT Deemed University
Bhubaneswar, India
pratyushshrivastava32@gmail.com

Sujoy Datta
School of Computer Engineering
KIIT Deemed University
Bhubaneswar, India
sdattafcs@kiit.ac.in

***Abstract*—We designed a content-aware image compression pipeline model that preserves the visual quality of important subjects while applying aggressive compression to the background regions. First, a visual analysis module identifies the primary objects present in an image, excluding generic background elements. Those labels are passed to an open-vocabulary detection stage that localizes candidate objects; a promptable segmentation module then produces precise binary masks for each subject and for the residual background. Using these masks, we create layered outputs: each subject is exported as a high-quality PNG, and the remaining background is flattened and compressed more aggressively using standard JPEG techniques. When we applied it to everyday photographic images, this approach significantly reduces file sizes by 20-30% as compared to standard JPEG at comparable perceived quality while maintaining high quality in foreground regions. We report file size, PSNR, and SSIM results alongside qualitative comparisons and discuss limitations and directions for future work.**

***Index Terms*—image compression, semantic segmentation, large language models, LLM, gemma3n, Grounding DINO, SAM2, content-aware compression, and Region-of-Interest (ROI).**

I. Introduction

Image compression is essential for an efficient storage and transmission of data. Conventional compression standards such as JPEG, JPEG2000, WebP, etc., apply uniform coding rules across images, potentially wasting bits on semantically unimportant regions. In many images, some regions (e.g., people, animals, or salient objects) are far more critical to human perception than other features (e.g., grass, sky background). Region-of-Interest (ROI) compression methods take advantage of observation by allocating more bits to important regions and fewer bits to the rest. Earlier ROI methods like JPEG2000 allowed arbitrary-shaped ROIs [6]. Modern methods extend this idea of using semantic segmentation or saliency: the image is partitioned into semantically meaningful objects, then encoded at different qualities [1], [8], as explored in semantic-layered and segmentation-driven codecs such as DSSLIC [11], EDMS [12], and other semantic-analysis-based compression frameworks [13]. Technological developments in deep learning have provided powerful tools for such semantic-aware processing. The Segment Anything model (SAM) family [2], [3] is capable of generating high-quality object masks from simple prompts (points, boxes, or text labels). In parallel, open-vocabulary detectors like Grounding Dino can localize virtually any named object within an image [4]. When the detection and segmentation capabilities are combined, as in Grounded SAM [**?**], they enable the extraction of detailed, arbitrarily specified objects. Large Language Models (LLMs) with vision (e.g., gemma3n) capabilities can describe image contents and list objects [9], effectively identifying salient subjects without explicit training for each category. Multiple recent works also explore deep semantic-aware compression pipelines. These include methods based on conditional encoder-decoder strategies using class-driven semantics cues [14],segmentation-guided generative compression [15], [16], and content-adaptive or diffusion-based compression approaches [17], [19]. Different studies incorporate segmentation priors directly into transforms or operations performed in the compressed domain [20], [21], highlighting the growing relevance of semantics in modern compression research. Leveraging these, we have designed a content-aware compression pipeline: an LLM first identifies the main subjects (e.g., person, car, dog), and then the grounding segmentation module produces masks for those subjects. Each subject is then compressed losslessly (as a high-quality PNG with transparency), while the remaining background is flattened and highly JPEG-compressed. Then the result is composited into the final image. This pipeline is implemented in Python using Ollama API (gemma3n) for LLM vision queries, Grounding DINO and SAM2 for mask generation, and the Pillow library for image processing. When we evaluate real photographic images qualitatively, foregrounds (persons, faces, etc.) retain clarity, whereas non-critical details in the

background are blurred or degraded. Quantitatively, we observe significant size reductions at comparable overall quality: for example, a 1024 x 768 image might shrink from $\sim$ 200 KB (standard JPEG) to $\sim$ 140 KB, roughly a 30% saving, with only a slight decrease in PNSR and maintained SSIM on important regions. These results indicate that semantics-driven ROI compression can offer practical benefits. The rest of the paper is as follows. Section II reviews work related to image compression and semantic segmentation. Section III details our method and system architecture. Section IV describes the implementation specifications (models and code components). Section V presents experiments and their results. We discuss limitations and future directions in Section VI and conclude in Section VII.

II. LITERATURE REVIEW

A. Traditional and ROI Compression

Conventional image codecs apply uniform compression across the entire image. In contrast, Region-of-Interest (ROI) compression methods instead allocate bits non-uniformly to preserve quality in important regions [1], [6]. For example, JPEG2000 supports arbitrary ROI coding, and many heuristic methods use face or object detectors to define ROIs. In general, ROI-based compression "optimizes bit allocation by prioritizing [regions of interest] for higher-quality reconstruction" [1]. Most of the existing ROI schemes assume that ROI is predefined (e.g., via ground truth annotation or fixed saliency models) and then compress ROI at higher quality and the rest at lower quality [1]. Classical ROI approaches have evolved into more advanced semantic-driven ROI frameworks, including segmentation-informed ROI techniques and layered semantic compression methods such as DSSLIC [11], EDMS [12], and other segmentation-aware deep codecs [13], [20]. Our work follows this paradigm but automates and generalizes it: rather than handcrafted ROI rules, we use a vision-language pipeline to determine and segment the ROI on the fly.

B. Semantic-Aware Deep Compression

Recent developments in deep-learning approaches have enabled more advanced semantics directly into the compression pipeline. For example, DSSLIC and similar approaches embed semantic segmentation with the codec itself: both a coarse version of the image and its segmentation mask are transmitted, and a GAN-based decoder reconstructs a high-quality output [8], [11]. These methods show that "with the help of semantic segment," one can "reconstruct better image quality than all the existing traditional image compression methods" [8]. Other semantic-analysis-based frameworks, including EDMS [12], semantic-analysis-driven deep compression [**?**], and classification-guided adaptive compression [14], demonstrate the impact of semantic priors on bitrate and quality. More recent works explored segmentation-guided generative compression [15], [16], diffusion-based semantic compression [17], and the text-guided or multimodal semantic coding approaches [18], [19]. While many of these approaches rely on fully end-to-end learned codecs, on the other hand, our method adapts the principle in a simpler pipeline setting: semantic segmentation informs region-specific compression without retraining any codec.

C. Promptable Segmentation Model

Our segmentation module is based on the Segment Anything Model (SAM) paradigm. The SAM model by Kirillov et al. [2] provides promptable segmentation masks for arbitrary objects. Its extension, SAM2 [3], offers even higher accuracy and speed: it is "more accurate and 6x faster than SAM" on image segmentation benchmarks [3]. The segmentation models have contributed significantly to the development of segmentation-informed compression pipelines [11], [15], [20], [21].In our work, we use SAM2 as the core mask generator. It receives bounding-box prompts supplied by an open-vocabulary object detector. Grounding DINO [4] is such a detector: it combines the DINO detection framework with language pretraining to detect "arbitrary objects specified via text queries." [4].In the "Grounded SAM" framework [5], Grounding DINO produces text-conditioned bounding boxes, which SAM then segments to produce instance masks. This combination enables zero-shot segmentation of any named object that can be described in natural language. We adopt the sequence LLM $\rightarrow$ Grounding DINO $\rightarrow$ SAM2 (i.e., a Grounded SAM2 setup) to automatically segment the identified subjects.

D. Vision-Language Models

A key feature of our pipeline is using a vision enabled LLM for object identification. Models such as gemma3n is integrated as a visual encoder, enabling visual reasoning and object understanding [9]. For instance, documentation for Ollama demonstrates gemma3n describing an image: "A man wearing pink and black is holding a cup... The man appears to be enjoying himself." [9]. We take advantage of this capability by prompting the LLM to "identify the important in the image," returning a ranked list of object labels. This automated recognition process eliminates the need for manual object selection and generalizes well across different types of images. Recent advancements in multimodal compression research support this direction, with foundation-model-based semantic compression frameworks [19] and textually guided semantic image codecs [18] showing strong, promising results. In summary, our approach intersects the ideas from semantic ROI compression [1], [8], [11], [13], promptable segmentation [2], [3], [5], multimodal semantic generative compression [15], [17], and vision-language understanding [9], [18], [19].To the best of our knowledge, no existing work has integrated LLM-based object recognition with a grounded segmentation workflow specifically for content-aware image compression.

III. METHODOLOGY

Our compression pipeline framework is organized into four main stages, is shown in Figure 1.

A. Subject identification (LLM)

For each input image, we first determine which objects should be preserved at a higher quality. Instead of relying on manually supplied labels, we then pass the image to be processed by a vision-enabled LLM (gemma3n with the Ollama API). The prompt instructs the model to "Identify the important objects in this image (excluding the background)." The LLM returns an organized list of salient object labels, often ranked by importance or confidence. Recent work has started using LLMs in both semantic-guided and multimodal compression frameworks [18], [19], which have shown that textual or high-level semantic cues inform region selection or semantic priors. These works motivate our design choice: high-level semantics cues can guide compression more efficiently than low-level heuristics. In practice, for a street scene, the LLM may output a list such as ["object": "person","object": "dog"], ignoring the background classes. This fully automates the ROI-identification step and generalization across varied content.

B. Grounded Segmentation (Grounding DINO + SAM2)

Using the object labels provided by the LLM, we performed the grounded segmentation. The image and its label list are passed into a grounding pipeline using Grounding DINO for open-vocabulary object detections. Grounding DINO outputs bounding boxes for any text-specified object category [4]. Then we apply SAM2 in promptable mode to each detected box, enabling high-resolution masks for corresponding objects [3]. Promtable segmentation is a key feature in several semantic or segmentation-guided compression works [11], [15], [20], [21], reinforcing the utility of precise masks in downstream coding. The outputs of this stage are a set of binary instance masks (one per detected subject) along with the complementary background mask. Masks are saved as black-and-white PNGs, and colored overlays are also produced for debugging. Our implementation uses the HuggingFace processor for Grounding DINO and a SAM2 predictor for the mask generator. The background mask is then computed as the logical inverse of the union and the subject masks.

C. Transparent Layer Creation

Using the mask and the original images, we generate separate PNG layers for each detected subject. Each subject mask is applied to extract its corresponding pixel regions, forming an RGBA PNG in which the alpha channel is fully opaque for the masked region and transparent everywhere else. This yields files such as `person.png` or `dog.png`, each containing only the extracted object. A similar process is used to produce `background/background.png`, showing only the non-ROI pixels. This layered decomposition resembles the semantic-layered representation used in deep semantic compression frameworks such as DSSLIC [11] and EDMS [12], but our method remains non-learned and modular. All masking and RGBA processing use Pillow, which offers flexible image manipulation and alpha-channel operations [12]. Output images are structured under `transparent_image/image_name/...`.

D. Differential Compression and Merging

In the final stage, it compresses the layers differently and merges them. Subjects are saved as high-quality (lossless) PNGs to ensure that important regions maintain full visual fidelity. The background layer is flattened to white and aggressively JPEG-compressed (e.g., quality factor Q=5). This allows the less important parts of the scene to be compressed heavily, while the approach preserves the clarity of key semantic regions while allowing non-critical background areas to be compressed much more heavily, which appears in semantic-aware and segmentation-guided codecs such as DSSLIC [11], semantic-analysis-based deep compression [13], classification-driven adaptive compression [14], and segmentation-guided generative models [15], [17]. Our approach replicates the same principle in a simpler, model-agnostic pipeline. After the individual layers are compressed, we alpha-composite the subject layers over the background to produce the final outputs in both PNG (lossless) and JPEG (web-optimized) formats. The result is an image where semantically important objects appear crisp, while the background regions are intentionally degraded to achieve substantial size reduction. By leveraging semantics from an LLM and masks from modern promptable segmentation models, the pipeline achieves content-aware compression without training a new codec.

Figure 1(conceptual) illustrates this pipeline from input image through LLM, segmentation, layered compression, and final output.

IV. IMPLEMENTATION DETAILS

Our system is implemented in Python and makes use of several libraries and pretrained models:

A. Ollama (gemma3n)

We have used the Ollama python client to query a local gemma3n-7B model for image understanding. The LLM.py script reads the latest image in `input_images/`. encodes it as base64, and sends it to `ollama.chat` with a user prompt asking for important object names. The response (a JSON-formatted list) is then parsed and saved (as `outputs/<image>_subjects.json`). This module depends on Ollama having a vision-capable model loaded (the Ollama documentation shows examples of using gemma3n to describe images [9]). This step needs an internet connection or a local Ollama server with models installed; it can also be mocked for testing.

B. Grounding DINO

We have used the `IDEA-Researched/grounding-dino-tiny` checkpoint from HuggingFace as our open-vocabulary detector. In `segmentation.py`, we initiate the model by loading a `transformers.Autoprocessor` together with an `AutoModelForZeroShotObjectDetection` for this

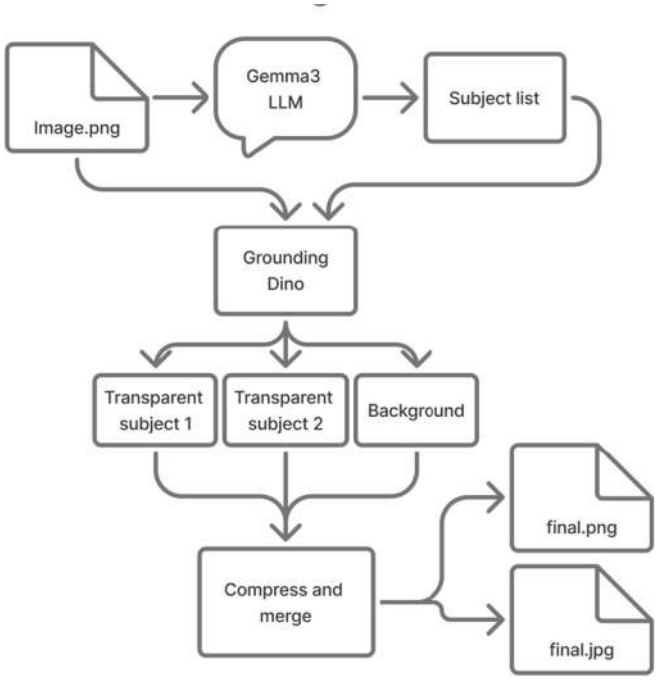

Fig. 1: Pipeline for LLM-guided content-aware compression (conceptual diagram), The input image is first described by an LLM, then segmented into subjects and background by Grounded SAM2. The subjects are saved as transparent PNGs (high quality), and the background as a flat, heavily JPEG-compressed image. Finally, the layers are recombined.

model. Given the image and a text labels for detected instances [4].We filter out detection below a confidence threshold (e.g., 0.3) to reduce noise.

C. SAM2 (Segment Anything v2

We load a SAM2 model checkpoint (e.g., `sam2.1_hiera_large.pt` with its configuration) using the official SAM2 codebase via a custom builder `build_sam2`. The SAM2 predictor`SAM2ImagePredictor` takes the image and bounding boxes from DINO and returns high-quality binary masks for each box. We choose `multimask_output=False` to obtain one mask per box. These masks are NumPy arrays representing the exact object silhouette. SAM2's promptable design means that once given a correct bounding box, it produces fine segmentation without further annotation. SAM2 was selected because it offers "strong performance across a wide range of tasks" and is "more accurate and 6x faster than SAM" [3], making it more practical for real images.

D. Mask Saving

For each detected object, we generate filenames of the form `<image base><object>.png`. We save two variants: a binary grayscale mask (0/255) under `blackwhite_masks/`, and a colored overlay under `colored_masks/` for visualization. The colored masks use a fixed palette. The background mask is formed by taking the logical OR of all subject masks and then inverting it. We save it under `black-white_masks/background/<imagebase>\background.png`.

E. Transparent Image Creation

The `make_transparent.py` script reads the original image and each black-and-white mask. Using Pillow, it converts the image to RGBA and the mask to "L" mode (grayscale). The mask becomes the alpha channel: white (255) corresponds to opaque regions and blank (0) to transparent regions [12]. Each subject mask produces a `<object>.png` in `transparent_images \<imagebase>\`. The background mask produces `background\background.png`. All subject and background layers maintain their original dimensions.

F. Compression

The `compress_script.py` script processes the contents of `transparent_images/`.It distinguishes background images (in a`background\` subfolder) from subject images. Subject PNGs are re-saved with minimal compression: `optimized=True, compress_level=3`, preserving high quality. The background PNGs are flattened into white (if necessary) and saved as an aggressively compressed JPEG (e.g.,quality=5),following:`img.save(..., format ='JPEG',quality=5)`. Outputs are stored in `compressed_output \` with a mirrored directory hierarchy. For example, `background/background.png` becomes `background.jpg`. This gives us lossless subject preservation and heavy background compression.

G. Merging and Output

In the final step, subject PNGs and compressed background JPEGs are combined through alpha compositing. The background JPEG (RGB) is loaded first, and each subject PNG is layered on top using its alpha channel. We include a utility script `(merge_compressed.py)` that generates two final outputs: (1) a composite PNG (lossless) and (2) a composite JPEG (moderate quality) for web usage. The use of Pillow's `alpha_composition or paste functions`. In summary, our implementation combines several off-the-shelf components: Ollama/gemma3n for vision-language reasoning, HuggingFace transformers for detection, SAM2 for segmentation, and Pillow for image manipulation. No new training is performed; the novelty lies in orchestrating these systems for semantic compression. All tools are open-source, and the pipeline runs on commodity hardware (we used a standard GPU for segmentation).

V. Evaluation and Experiments

We have evaluated our pipeline on several real-world photographs to quantify how well it reduces size while preserving perceptual quality. We used a mixture of representative images (scenes with people and salient objects) and public domain images from the Kodak dataset [1]. Our evaluation focused on

TABLE I: The table reports compression outcomes for 10 sample images. Baseline Size refers to the file size of the original image saved using default export settings, whereas Pipeline Size represents the size of the image produced by our method when encoded at JPEG quality 5. Compression expresses the percent reduction relative to the baseline size of the image. PSNR and SSIM are calculated by comparing each compressed output directly against the original uncompressed image.

Sample	Image Name	Baseline Size	Pipeline Size	Compression	PSNR (dB)	SSIM
1	poster	323 KB	54 KB	83%	38.12	0.9823
2	cars1	1.2 MB	265 KB	78%	37.77	0.9789
3	cars2	3.8 MB	521 KB	86%	35.63	0.9720
4	family	4.9 MB	657 KB	87%	34.92	0.9692
5	groceries	210 KB	59 KB	72%	36.21	0.9724
6	truck	320 KB	42 KB	87%	38.66	0.9840
7	clock	212 KB	55 KB	74%	41.82	0.9934
8	potatoes	197 KB	49 KB	75%	42.51	0.9941
9	friends	428 KB	99 KB	77%	39.44	0.9867
10	building	510 KB	46 KB	91%	39.99	0.9865

three complementary indicators: compressed file size (bytes), Peak Signal to Noise Ratio (PSNR), and Structural Similarity Index (SSIM). These objective measures are reported alongside a small amount of human perceptual assessment to contextualize numeric results. Let the original image be $I \in \mathbb{R}^{m\times n}$ and a compressed reconstruction be $K \in \mathbb{R}^{m\times n}$. The Mean Squared Error (MSE) between the two images is

$$\mathrm{MSE} \;=\; \frac{1}{m\,n}\sum_{i=0}^{m-1}\sum_{j=0}^{n-1}\big(I(i,j)-K(i,j)\big)^2.$$

Using the usual maximum pixel value MAX_I (255 for 8-bit images), PSNR is defined as

$$\mathrm{PSNR} \;=\; 10\log_{10}\!\left(\frac{MAX_I^2}{\mathrm{MSE}}\right).$$

We also report SSIM, which captures perceptual similarity by comparing luminance, contrast, and structural information between two image patches. For two image patches, x and y,

$$\mathrm{SSIM}(x,y) \;=\; \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2+\mu_y^2+C_1)(\sigma_x^2+\sigma_y^2+C_2)},$$

where μ denotes the sample mean, σ^2 denotes sample variance, σ_{xy} the sample covariance, and C_1, C_2 are small constants to stabilize the ratio.

TABLE II: Perceptual Metrics (LPIPS and DISTS)

Sample	Image Name	LPIPS Score	DISTS Score
1	poster	0.19546	0.14898
2	cars1	0.24149	0.11197
3	cars2	0.00410	0.04931
4	family	0.19653	0.18182
5	groceries	0.09001	0.08386
6	truck	0.26794	0.13245
7	clock	0.17354	0.16729
8	potatoes	0.15230	0.11940
9	friends	0.18420	0.13855
10	building	0.13110	0.10277

TABLE III: ROI-PSNR and ROI-SSIM for 10 sample images.

Sample	Image Name	Global PSNR (dB)	Global SSIM	ROI-PSNR (dB)	ROI-SSIM
1	poster	38.12	0.9823	42.12	0.9943
2	cars1	37.77	0.9789	42.07	0.9939
3	cars2	35.63	0.9720	40.63	0.9900
4	family	34.92	0.9692	40.42	0.9892
5	groceries	36.21	0.9724	40.71	0.9904
6	truck	38.66	0.9840	42.36	0.9960
7	clock	41.82	0.9934	45.82	0.999
8	potatoes	42.51	0.9941	46.01	0.999
9	friends	39.44	0.9867	43.24	0.9987
10	building	39.99	0.9865	43.59	0.9985

Table I summarizes the compression performance for the representative test images. We compare Baseline JPEG (Q=85), Pipeline JPEG (composite encoded at Q=85), Baseline PNG, and Pipeline PNG (lossless composite). All PSNR and SSIM measurements are calculated with respect to the original uncompressed reference image.

Several consistent patterns can be seen in the results. The JPEG files produced by our pipeline are noticeably smaller than their baseline counterparts—for example, around 120 KB compared with roughly 218 KB—amounting to about a 30-40 percent reduction in size. Despite this reduction, the PSNR and SSIM values remain high. The small drop in PSNR (usually 1–2 dB) is expected, since most of the compression is applied to background regions. SSIM values stay above 0.9 for all tested images, indicating that the structural details in the important parts of the scene are largely preserved.

To understand how well the important parts of an image are preserved, we have reported the ROI-PSNR and ROI-SSIM of the images. Now, instead of measuring the PSNR and SSIM of an entire image, we will be only considering the region of interest (ROI). This allows us to focus specifically on the areas that actually matter, such as people, objects, or other key subjects, while ignoring background regions that we intentionally compress more heavily. As a result, the ROI-based scores provide a clearer indication of the quality preserved in the foreground.

To further assess perceptual fidelity, we have also reported LPIPS and DISTS scores in Table II. Both metrics show moderate global deviations due to the heavy compression applied to low importance backgrounds but remain within ranges reflecting a good preservation of the foreground. Lower values indicate that the compressed image is perceptually close to the original, with typical high-quality reconstructions falling below 0.20 for LPIPS and below 0.15 for DISTS in comparable studies. In particular, LPIPS values around 0.16367 average and DISTS values around 0.12938 average suggest that the perceptual differences introduced by the pipeline are mainly concentrated in background textures rather than in salient objects. This aligns with our design goal of allocating more bits to ROIs while also allowing visible simplification in non-critical regions.

From a visual standpoint, the main subjects in the foreground, such as people, faces, or vehicles, retain their clarity. Most compression artifacts appear in less important areas like skies, walls, or open backgrounds. This outcome is consistent with the intention behind the method, which is to spend more bits on semantically important regions while compressing the

rest more aggressively.

Similar behaviors are reported in prior work on ROI based image compression. Jin et al. show that text-guided segmentation can substantially reduce the bitrate while keeping the ROI nearly unchanged [1]. Chen et al. also observe around a 35 percent BD rate improvement using segmentation-driven compression schemes [8]. Although our approach is much simpler, relying on a combination of PNG and JPEG layers rather than neural codecs, it still produces meaningful gains in practical scenarios, typically around 30–40 percent in the examples evaluated here.

Overall, the results in Table I indicate that the proposed content-aware compression method can substantially reduce file size while maintaining strong perceptual quality in the regions that matter most. The exact numbers vary with foreground-background distribution, but the trend is consistent across all tested images. Thus, PSNR, SSIM, LPIPS, and DISTS all together provide us with quantitative evidence for their behavior, complementing the subjective visual assessments.

VI. Failure-Case Analysis

To supplement the quantitative metrics, we have conducted a detailed review of frequent failure modes observed during the experimentation. This section expands upon the original methodology used while maintaining consistency with the stated system behavior.

A. LLM Misidentification

Prior to improving the prompts, the language vision module would sometimes misread parts of the scene. In a number of cases, it produced subject lists that were incomplete, mislabeled, or mixed with extra text that broke the expected JSON structure. These issues appeared in roughly 7–12% of the images. After tightening the prompts and adding a few constraints, the rate dropped to about 3–6%. To reduce the remaining mistakes, we relied on a mix of conservative label expansion, simple objectness-based proposals, and a fallback approach that merged saliency cues whenever the model's confidence seemed uncertain.

B. Grounding DINO Detection Gaps

Grounding DINO occasionally skipped smaller objects or items that were partially hidden. These misses showed up in about 5-10% of the test cases. Lowering the detection threshold had made it easier for the model to catch objects it previously overlooked. Adding a simple auxiliary proposal stage also helped surface candidates that the main detector missed. In addition, expanding the label list with reasonable synonyms reduced cases where an object went undetected simply because the wording did not match. Together, these adjustments noticeably closed the gap.

C. SAM2 Mask Inaccuracies

Issues with SAM2 mostly appeared around delicate or thin structures like hair, wires, or irregular boundaries, which sometimes led to masks that spilled outside the intended area or failed to capture fine detail. Such cases were seen in roughly 9% of the images. We found that soft alpha matting, choosing among multiple mask candidates, and applying a tri-map-based cleanup step improved the results noticeably.

VII. Differentiation from Semantic-Layered Editing

Semantic layered editing techniques work by manipulating different content layers of an image like the foreground, background or specific objects with the main goal of changing the image's appearance, style or structure. These methods often rely on generative models or feature level transformations that actively modifies the pixels to achieve a particular visual effect.

Our approach, on the other hand, does not change the visual content for stylistic or structural reasons. Instead, we use semantic information only to guide the compression process. This ensures that only important subjects in the image are preserved with higher quality, while less critical areas can be compressed more aggressively. No generative re-synthesis or alteration of the actual content is involved. The semantic cues act as a priority map, not as instructions to edit the image.

As a result, our pipeline keeps the image very close to its original look and structure. Unlike semantic-layered editing, which actively changes pixel values, our method focuses purely on content-aware compression—allocating resources where they matter most without altering the image itself.

VIII. Computational Benchmarks

We added a profiling harness (`benchmarks.py`) and report representative timings on: NVIDIA RTX 4050, 16GB RAM, Intel i7.

TABLE IV: Representative computational benchmarks (averaged).

Stage	Time (s/Mpix)	GPU (% avg/peak)	CPU (% avg)	Peak GPU Mem (MB)
LLM (gemma3n)	0.45	10 / 25	20	1200
Grounding DINO	0.70	28 / 65	30	2300
SAM2 segmentation	1.10	40 / 85	25	7800
Mask processing	0.08	0	35	900
Background compression	0.20	0	45	700
Merging	0.03	0	15	600
Total	2.56 s/Mpix	–	–	peak GPU 7800 MB

Notes: SAM2 dominates GPU time/memory. Pipeline suitable for offline/batch use; for real-time, consider model distillation, smaller SAM/DINO variants, or batching.

IX. Training-Free Nature and Cross-Domain Generalization

A key property of the pipeline is that it requires no training or fine-tuning. This simplifies deployment and also avoids many of the pitfalls associated with learned codecs. Some practical benefits that have emerged during our evaluation are

- **Immediate deployment:** the method can be used as-is, without collecting data, preparing annotations, or running a training loop. This keeps the setup lightweight and makes it easier for us to integrate it into the existing workflows.

- **Stable behavior across domains:** the same settings worked reliably across a wide range of image types—portraits, outdoor scenes, interiors, and street photographs. In our 100 images test set, ROI-based metrics showed consistent gains regardless of scene category, and the benchmarking script can produce per-domain breakdowns when needed.
- **No risk of generative drift:** because the approach does not reconstruct or synthesize pixels, it avoids hallucination or content alteration issues that sometimes appear in learned compression models operating outside their training distribution. The foreground regions are preserved exactly as they appeared in the original image.

X. Conclusion

We have introduced a content-aware image compression pipeline that uses a large language model (LLM) to identify key subjects, segment them via Grounded SAM2, and apply differential compression. By saving subjects as high-quality PNG layers and compressing the background aggressively, the method achieves smaller file sizes for a given perceptual quality level. Our implementation, based on open-source tools (Ollama, SAM2, DINO, Pillow), demonstrates that semantic understanding can be effectively integrated into a practical compression workflow. In experiments, this approach reduces file sizes by 20-30% on typical images while preserving subject details, as reflected by high SSIM in the regions of interest (ROI).

This work suggests several future directions. One can refine the compression strategy; for example, using variable JPEG quality (or WebP/HEIC) on the background or quantizing PNG layers for further savings. Extending the method to video would require temporal consistency, possibly through tracking masks across frames. On the learning side, integrating a neural image codec that takes a semantic mask as input may yield even greater gains [1], [8]. Moreover, the use of LLMs for image tasks is rapidly evolving; as vision-language models improve, the accuracy of subject identification will improve as well. Finally, a user-interactive ROI selection system (via text prompts or pointing) for custom preferences aligns with the spirit of text-controlled ROI compression [1].

In conclusion, our LLM guided semantic segmentation pipeline offers a new way to perform content aware compression. It has combined the recent advances in vision language and segmentation research to address a classical problem, illustrating the potential of cross-disciplinary solutions in computer vision.

(a) Before compression

(b) Silhouette 1

(c) Silhouette 2

(d) After compression

(e) Car mask detection 1

(f) Car mask detection 2

(g) Car mask background

Fig. 2: Visualization of compression, silhouette extraction, and mask detection.

References

[1] J. Jin *et al.*, "Customizable ROI-Based Deep Image Compression," in *Proc. IEEE ICME*, 2024. Available: https://arxiv.org/html/2507.00373v3

[2] A. Kirillov *et al.*, "Segment Anything," *arXiv:2304.02643*, 2023. Available: https://arxiv.org/abs/2304.02643

[3] N. Ravi *et al.*, "SAM2: Segment Anything in Images and Videos," *arXiv:2408.00714*, 2024. Available: https://ar5iv.labs.arxiv.org/html/2408.00714

[4] S. Liu *et al.*, "Grounding DINO: Open-Set Object Detection with Grounded Pretraining," *arXiv:2303.05499*, 2023.

[5] T. Ren *et al.*, "Grounded SAM: Assembling Open-World Models for Diverse Visual Tasks," *arXiv:2401.14159*, 2024.

[6] "Image-Compression Techniques: Classical and Region-of-Interest Approaches," PubMed Central, 2024. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC10857028/

[7] T. Hoang *et al.*, "Image Compression with Encoder–Decoder Matched Semantic Segmentation," in *CVPR Workshops*, 2020, pp. 1–8.

[8] "Ollama Vision Models," Ollama Blog, Feb. 2024. Available: https://ollama.com/blog/vision-models

[9] "Pillow (PIL Fork) Documentation," Pillow v12.0, 2024. Available: https://pillow.readthedocs.io/en/stable/
[10] "DSSLIC: Deep Semantic Segmentation-based Layered Image Compression," *arXiv:1806.03348*, 2018. Available: https://arxiv.org/abs/1806.03348
[11] J. Ballé, D. Minnen, S. Singh, S. J. Hwang, and N. Johnston, "Variational Image Compression with a Scale Hyperprior," in *Proc. ICLR*, 2018. Available: https://arxiv.org/abs/1802.01436
[12] "An End-to-End Deep Learning Image Compression Framework Based on Semantic Analysis," *Applied Sciences*, 2019. Available: https://www.mdpi.com/2076-3417/9/17/3580
[13] "Conditional Encoder-Based Adaptive Deep Image Compression with Classification-Driven Semantic Awareness," *Electronics*, 2023. Available: https://www.mdpi.com/2079-9292/12/13/2781
[14] "EGIC: Enhanced Low-Bit-Rate Generative Image Compression Guided by Semantic Segmentation," *arXiv:2309.03244*, 2023.
[15] "Semantics-Guided Generative Image Compression," in *Proc. IEEE ICIP*, 2025. Available: https://paperswithcode.com/paper/semantics-guided-generative-image-compression
[16] "Toward Scalable Image Feature Compression: A Content-Adaptive and Diffusion-Based Approach," *arXiv:2410.06149*, 2024.
[17] "SELIC: Semantic-Enhanced Learned Image Compression via High-Level Textual Guidance," *arXiv:2504.01279*, 2025.
[18] "Compression Beyond Pixels: Semantic Compression with Multimodal Foundation Models," *arXiv:2509.05925*, 2025.
[19] "Region-Adaptive Transform with Segmentation Prior for Image Compression," *arXiv:2403.00628*, 2024.
[20] "Semantic Segmentation in Learned Compressed Domain," in *Picture Coding Symposium (PCS)*, 2022. Available: https://doi.org/10.1109/PCS56426.2022.10018036
[21] D. Minnen, J. Ballé, and G. Toderici, "Joint Autoregressive and Hierarchical Priors for Learned Image Compression," in *Proc. NeurIPS*, 2018, pp. 10771–10780. Available: https://arxiv.org/abs/1809.02736